

Genome polarisation with diemr

Natália Martínková

Last updated: 2026-01-02

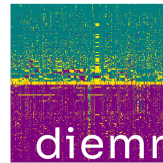
Contents

1	Introduction	5
1.1	How to cite	6
2	Quick start	7
3	Input data	11
3.1	Multiple compartments with different ploidies	11
4	Importing data for genome polarisation	15
4.1	Conversion from VCF format	15
4.2	Principle of conversion to diem format	17
4.3	Variant call format	19
4.4	Polyploid data	20
5	Understanding genome polarisation output files	21
5.1	Obligatory outputs	21
5.2	Optional outputs	23
6	Visualisation of polarised genotypes	29
6.1	Plotting polarised genotypes	29
6.2	Circular (iris) plot	34
7	Filtering sites by diagnostic index	37
7.1	Selecting the filtering threshold	40
7.2	Biological signal insights	42

8	Smoothing polarised genotypes	45
8.1	Smoothing across mapped distances	45
8.2	Interpretation and downstream use	48
9	Ploidy-aware genome polarisation	53
9.1	Mapping compartments and their ploidy	53
9.2	Hybrid index under varying ploidy	55
9.3	Implementation	55
9.4	Interpretation considerations	57

Chapter 1

Introduction



Genome polarisation is a genome-painting approach based on the likelihood-based diagnostic index expectation maximisation (*diem*) algorithm (Baird et al., 2023). It identifies which alleles of single-nucleotide variants (SNV) belong to either side of a barrier to gene flow, co-estimating both the assignment of individuals to a barrier side and the diagnosticity of each marker, meaning how consistently individuals on one side are homozygous for the allele associated with that side.

By inferring which parts of the genome correspond to each parental lineage, genome polarisation provides a direct view of **how barriers to gene flow shape genomic architecture**. It can detect and quantify hybridisation, distinguish introgressed from non-introgressed regions, and reveal how species boundaries evolve during speciation and diversification. Compared with methods such as STRUCTURE, ADMIXTURE, or PCA, which summarise population structure statistically, genome polarisation explicitly identifies the genomic segments that define the divergence between taxa or lineages.

The diagnostic index computed by *diem* **highlights markers that are most informative** for the primary axis of genetic differentiation. These diagnostic loci can then be used to describe patterns of hybridisation, assess barrier strength, or visualise the genomic distribution of ancestry.

This book provides a step-by-step guide to performing genome polarisation analyses in R using the `diemr` package, also available from CRAN. The package includes functions for input validation, file-format conversion, visualisation, and

diagnostic summaries, with examples based on typical genomic datasets such as variant call format (VCF) files and SNP matrices. By the end of the book, users will be able to run the complete analysis workflow, interpret its outputs, and generate graphical representations of genome polarisation.

The *diem* algorithm itself is also implemented in **Mathematica** and **Python**, available [here](#), allowing reproducibility and interoperability across analytical environments.

1.1 How to cite

If you use the *diemr* package or this documentation in your work, please cite:

Baird, S. J. E., J. Petružela, I. Jaroň, P. Škrabánek, and N. Martínková. 2023. *Genome polarisation for detecting barriers to geneflow*. *Methods in Ecology and Evolution* 14: 512–28. <https://doi.org/10.1111/2041-210X.14010>.

and this online documentation for the **diemr** package:

Martínková, N. 2025. *Genome polarisation with diemr*. Bookdown online documentation. Available at: <https://nmartinkova.github.io/genome-polarisation/>.

Chapter 2

Quick start

Natália Martínková and Stuart J. E. Baird

The package `diemr` incorporates the diagnostic index expectation maximisation algorithm used to estimate which genomic alleles belong to which side of a barrier to gene flow. To start using `diemr`, load the package or install it from CRAN if it is not yet available:

```
# Attempt to load a package, if the package was not available, install and load
if(!require("diemr", character.only = TRUE)){
  install.packages("diemr", dependencies = TRUE)
  library("diemr", character.only = TRUE)
}
```

Next, assemble paths to all files containing the data to be used by `diemr`. Here, we will use a tiny example dataset that is included in the package for illustrating the analysis workflow. A good practice is to check that all files contain data in the correct format for all individuals and markers. Additionally, the analysis will need a list with **ploidies for all genomic compartments and individuals**, and a vector with **indices of samples** that will be included in the analysis.

```
filepaths <- system.file("extdata", "data7x3.txt", package = "diemr")
# Analysing six individuals
samples <- 1:6
# Assuming diploid markers of all individuals
nIndividuals <- nchar(readLines(filepaths[1])[1]) - 1)
ploidies = rep(list(rep(2, nIndividuals), length(filepaths))
CheckDiemFormat(files = filepaths,
                  ChosenInds = samples,
                  ploidy = ploidies)
```

```
# File check passed: TRUE
# Ploidy check passed: TRUE
```

If the `CheckDienFormat()` function fails, work through the error messages and fix the stored input files accordingly. The algorithm repeatedly accesses data from the harddisk, so seeing the passed file and variable check prior to the analysis is critical.

To estimate the marker polarities, their diagnostic indices and support, run the function `diem()` with default settings. Here, we have only one file with data, so parallelisation is unnecessary, and we set `nCores = 1`. **The Windows operating system only allows for `nCores = 1`.** Other operating systems can process multiple genomic compartments (e.g. chromosomes) in parallel, the analysis of different genomic compartment files running on multiple processors.

```
res <- diem(files = filepaths,
            ploidy = ploidies,
            markerPolarity = list(c(FALSE, FALSE, TRUE)),
            ChosenInds = samples,
            nCores = 1)
```

The result is a list, where the element `res$DI` contains a table with marker polarities, their diagnostic indices and support.

```
res$DI
#   newPolarity      DI   Support Marker
# 1      FALSE -4.872256 15.930181      1
# 2       TRUE -3.520647 18.633399      2
# 3       TRUE -13.274822  6.130625      3
```

The column `newPolarity` means that marker 1 should be imported for subsequent analyses as is, and markers 2 and 3 should be imported with 0 replaced with 2 and 2 replaced with 0 (hereafter ‘flipped’ $0 \leftrightarrow 2$). The marker 3 has the lowest diagnostic index and low support, indicating that the genotypes scored at this marker are poorly related to the barrier to geneflow arbitrated by the data.

With the marker polarities optimised to detect a barrier to geneflow, a plot of the polarised genome will show how genomic regions cross the barrier. First, the genotypes need to be imported with optimal marker polarities. Second, individual hybrid indices need to be calculated from the polarised genotypes. And last, the data will be plotted as a raster image.

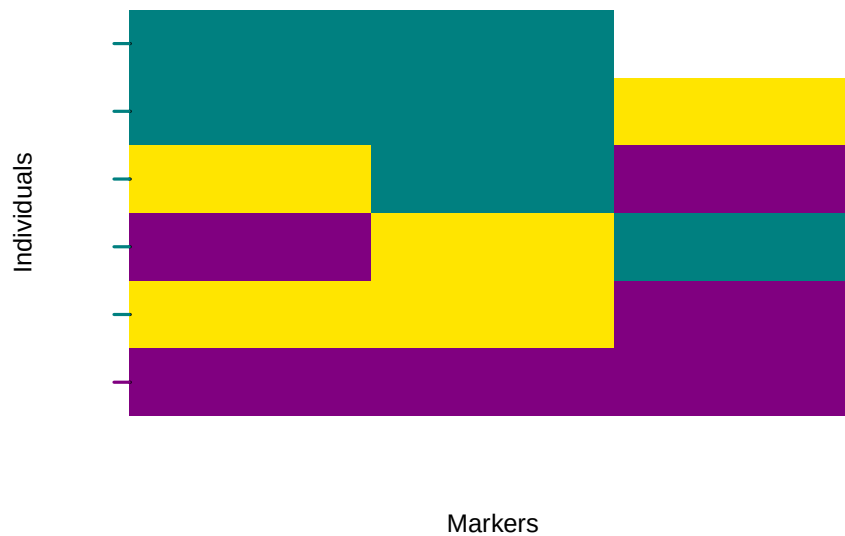

```

genotypes <- importPolarized(file = filepaths,
                             changePolarity = res$markerPolarity,
                             ChosenInds = samples)

HI <- hybridIndex(res$I4)

plotPolarized(genotypes = genotypes,
              HI = HI[samples])

```



CAUTION: This is just a *quick start* to get you started! For real datasets you will use the diagnostic index (DI) to filter the full set of sites you have analysed with `diem` in order to plot only those markers relevant to any barrier detected in the analysis. See **Chapter 7** for guidelines.

Chapter 3

Input data

Natália Martínková and Stuart J. E. Baird

The `diemr` package uses a concise genome representation. Let's have a small dataset of three markers genotyped for seven individuals.

```
S0011222  
S1210001  
S02221U0
```

The genotypes encoded as 0 represent homozygotes for an allele attributed to barrier side A, 1 are heterozygous genotypes, 2 are homozygotes for another allele, attributed to barrier side B, and U (symbol “_” is also allowed) represents an unknown state or a third (fourth) allele. The power of `diem` lies in the assurance that the user does not need to determine the true assignment to barrier sides A and B before the analysis and the specific genotypes encoded as 0 and 2 respectively can be arbitrary.

The leading `S` on each line of the input file ensures that the marker genotypes are read in as a string on all operating systems. The `S` is dropped during import of the genotypes, and the dataset is imported as a character matrix of all sites.

3.1 Multiple compartments with different ploidies

Some genomic compartments differ between individuals in their ploidy. For example, markers located on chromosome X in mammals will be diploid in females, but haploid in males. Ploidy differences between individuals influence calculation of the hybrid index, which in turn has an effect on the *diem* analysis.

To set up the *diem* analysis with multiple compartments, the markers with different individual ploidies must be stored in separate files. The file analysed in the **Quick start** chapter could contain markers from autosomes and an additional file will contain markers from an X chromosome, with individuals 2 and 6 being males. The respective ploidies for the second genomic compartment will be `c(2, 1, 2, 2, 2, 1, 2)`.

Arguments `files` and `ploidy` will need to reflect the information, taking care that the order of filenames corresponds to the order of elements in the list of ploidies. *diem* cannot check that the order of the elements is correct, only that the information is in the correct format.

```
filepaths2 <- c(system.file("extdata", "data7x3.txt", package = "diemr"),
               system.file("extdata", "data7x10.txt", package = "diemr"))

ploidies2 <- list(rep(2, 7),
                 c(2, 1, 2, 2, 2, 1, 2))

CheckDiemFormat(files = filepaths2,
                 ChosenInds = samples,
                 ploidy = ploidy2)
# File check passed: TRUE
# Ploidy check passed: TRUE

# Set random seed for reproducibility of null polarities (optional)
set.seed(39583782)

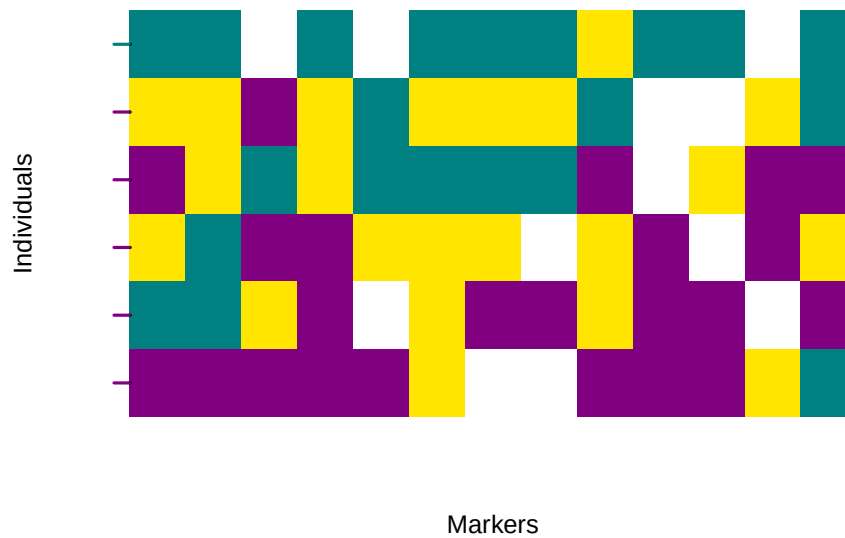
# Run diem
res2 <- diem(files = filepaths2,
             ploidy = ploidy2,
             markerPolarity = FALSE,
             ChosenInds = samples,
             nCores = 1)
```

Plotting polarised genomes from multiple compartments requires separate import of the compartment data. The polarities in the `res2$markerPolarity` element are combined across all compartments, and extracting them requires knowledge of the number of markers in each compartment. Alternatively, the marker polarities from each compartment can be extracted from the list in the `res2$PolarityChanges` element.

```
# Import each compartment into a list
genotypes2 <- importPolarized(file = filepaths2,
                             changePolarity = res2$markerPolarity,
                             ChosenInds = samples)
```

```
# Load individual hybrid indices from a stored file
h2 <- hybridIndex("HIwithOptimalPolarities.txt")

# Plot the polarised genotypes
plotPolarized(genotypes = genotypes2,
              HI = h2[samples])
```



See Chapter 9 for extended explanation of ploidy-aware genome polarisation.

Chapter 4

Importing data for genome polarisation

Natália Martínková and Stuart J. E. Baird

4.1 Conversion from VCF format

The package `diemr` provides a function `vcf2diem`, which converts diploid genotypes in a VCF file (optionally, the VCF file can be gzipped) or an `vcfR` object loaded into R to a file with format required by `diem`.

Genome polarisation requires variant sites to be processed in physical order along chromosomes. If your pipeline contained sorting, no action is needed. Otherwise, guidelines for correcting site order are provided in the hidden section below.

Sorting VCF files before genome polarisation

4.1.1 Sorting VCF files before genome polarisation

Many tools produce ordered VCF files by default, but others may output scaffolds or contigs in an arbitrary sequence, or return variants in the order they were discovered rather than their coordinates.

Before importing any VCF file into `diemr`, confirm that sites are sorted by chromosome and position.

Sorting matters for three reasons:

- **Spatial summaries and genome-wide visualisations** depend on correct genomic order (Chapter 6). Hybrid index interpretation is unaffected, but positional summaries are not.
- **Marker axis plotting** (Chapter 6.1.1.1) requires positions to increase monotonically. Unsorted input causes axes to wrap or jump.
- **Window-based smoothing** (Chapter 8) relies on physical proximity. Unsorted sites disrupt neighbourhood windows and distort ancestry patterns.

To ensure that the VCF is correctly sorted, use `bcftools sort`, which respects contig order in the header and handles large files efficiently.

Example (recommended):

```
bcftools sort input.vcf -Oz -o sorted.vcf.gz
tabix -p vcf sorted.vcf.gz
```

The VCF files can also be sorted with tools such as `vcf-sort` (from `vcftools`) or a plain UNIX `sort`, but these methods rely on lexicographic chromosome order and may mis-order contigs unless a contig header is present. For reproducible workflows, always prefer `bcftools sort`.

Once the VCF is sorted and indexed, it is ready for `vcf2diem` conversion.

The number of chunks the SNVs are divided into matters for parallelisation of `diem` analyses and import of polarised genotypes. Therefore, decide whether all sites in the VCF file should be polarised from one file, or whether the analysis will benefit from parallelisation. Parallelisation is available on UNIX-based platforms, and we advise to use it for large datasets. `diem` itself can also work in parallel on multiple data *compartments*. VCF files and *compartments* can correspond to the same thing or you can concatenate/split `vcf2diem` outputs to produce `diem` input *compartments*. Parallelisation is most efficient when **chunks contain the same number of SNVs**.

```
# Path to the vcf file
inputfile <- system.file("extdata", "myotis.vcf", package = "diemr")
# File name for the output
# If working directory does not have writing privileges,
# the filename must include a path to a suitable folder
outputfile <- "myotis"
# Convert the vcf file to two diem files
vcf2diem(SNP = inputfile, filename = outputfile, chunk = 2)
# Expecting to include 11 sites per diem file.
# If you expect more sites in the file, provide suitable chunk size.
# Done with chunk 1
```


The `vcf2diem` function converts diploid calls into the `diem` site encoding. The order of sites in the output matches the original VCF file (but see chapter Sites not informative for genome polarisation).

4.2 Principle of conversion to diem format

The `diemr` package uses a concise genome representation, differentiating homozygotes for the two most frequent alleles at each site, and their respective heterozygotes (mixed state of the two most common alleles). All other genotypes represent an unknown state. Specifically, the genotypes encoded as 0 represent homozygotes for one of the two most frequent alleles, 1 are heterozygous (mixed state) genotypes for the two most frequent alleles, 2 are homozygotes for the other allele, and U (symbol “_” is also allowed) is an unknown state that can be a missing genotype or a genotype that includes a third (or a fourth) allele.

4.2.1 Invariant sites and indels

What logically follows from this description is that genome polarisation is meaningful only for variant sites (we call variant sites with high diagnostic index after genome polarisation markers). Invariant sites will have **low support** and, though they will not change the `diem` outcome, they will slow down convergence and obscure the nature of change between taxa. During `diem` iterations their obscuring effect is removed by rescaling the hybrid indices onto $[0, 1]$

Including invariant sites in genome polarisation is acceptable, and likely unavoidable. A proportion of truly invariant sites will be mis-diagnosed as variant due to sequencing errors.

At this time, genome polarisation uses variant sites where alleles differ in nucleotide substitutions. Insertions or deletions (indels) are not allowed to be classified among the most frequent alleles in the `vcf2diem` function.

4.2.2 Sites not informative for genome polarisation

Along side invariant sites, some variant sites are also uninformative for genome polarisation. Sites that include only missing data and heterozygous genotypes are unpolarisable. In general applications, one might wish to exclude sites that are only included in a `vcf` due to the presence of a single heterozygous individual.

Uninformative sites will have support equal to 0, and their polarity will thus be equal to the null polarity. Including uninformative sites has similar consequences to including invariant sites. When a user wishes to retain such sites, we advice to then filter polarised markers with high support (or simply high diagnostic index, which is strongly correlated with support) for subsequent analyses and interpretation.

4.2.3 Haploid data

Haploid data, including sites on sex chromosomes of the heterogametic sex, are usually shown as sequence alignments. To illustrate the principle of genome representation used in **diem**, let's have an 8bp-long alignment with five individuals.

```

5 8
ind1 AACCTTGG
ind2 TACGATGG
ind3 TACC-TGT
ind4 CACGTTGG
ind5 AACCTTGT

```

We can see that the example alignment contains four variant sites at sites 1, 4, 5, and 8. Site 5 contains a deletion in ind3, while other sites vary in different nucleotide substitutions. In site 1, two individuals have an allele A, two individuals have an allele T and one has a C. This site can be resolved in terms of two most frequent alleles A and T, where the individual ind4 has a third allele and is therefore considered as having an unknown genomic state. To convert the site into **diem** format, we must make a decision on which allele will be encoded as 0. The choice is arbitrary. Good practice is to **keep a record of genotype encoding**. Let's decide that all alleles in ind1 will be encoded as 0. Site 1 will then be 022_0.

Contrary to DNA sequence alignments, diem format transposes the data. Rows in diem input files are sites and columns represent individual samples. The diem output for the example alignment will thus have four lines representing the four variant sites, and five columns representing the individuals.

```

Site1 022_0
Site4 02020
Site5 02_00
Site8 00202

```

The *diem* format does not store information on site identification or location. This metadata must be saved separately in a 'bed-like' file (with at minimum the scaffold and position for each site). **diem** uses a prefix **S** at line starts to ensure that all operating systems always read genotypes as strings without attempting to convert them into numbers. Encoding of sites 4 and 8 shown above is at risk of being interpreted as numeric. The final file in **diem** format will be:

```

S022_0
S02020
S02_00
S00202

```

4.2.4 Diploid data

4.2.4.1 Heterozygotes in DNA sequence alignments

Diploid sites in DNA sequence alignment can be resolved in an analogous way to haploid sites. Heterozygotes in alignments can be identified according to their IUPAC DNA ambiguity codes. For example, code W may represent an accepted heterozygote in Site 1, where the most frequent alleles are A and T.

4.3 Variant call format

Variant sites in genomic context will be most often identified using variant callers from sequence reads mapped onto a reference. Such data are stored or can be converted to variant call format (VCF). We implemented conversion of VCF files to diem format for the VCF version 4.2 in function `vcf2diem`. The function resolves indels and sites with more than two alleles to obtain biallelic genotypes for all sites included in the original VCF file.

1. Sites with the reference allele (column REF) containing insertions are set as unknown.
2. Sites with the alternative allele (column ALT) containing only indels are set as unknown.
3. Indels in the alternative alleles are set as unknown.
4. Overall allele counts in sites with more than two alleles representing substitutions are calculated and the two most frequent alleles are selected. Ties are (arbitrarily) resolved according to the allele order in the VCF file.
5. Third and fourth most frequent substitutions are set as unknown.

The conversion might result in some sites no longer being polymorphic. We advise checking how frequent invariant sites are after conversion.

```
# Import the first converted file for all individuals
# and without changing site polarity
myotis <- importPolarized("myotis-01.txt",
                        changePolarity = rep(FALSE, 11),
                        ChosenInds = 1:14)

# Check whether a site includes heterozygotes
# or that there is more than one genotype
apply(myotis, MARGIN = 2,
      FUN = \(x) any(grepl("1", x)) | sum(levels(factor(x)) %in% c("0", "1", "2")) > 1)
#   m1   m2   m3   m4   m5   m6   m7   m8   m9  m10  m11
# FALSE FALSE FALSE FALSE TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
```

Starting from version 1.2.2, `vcf2diem` removes sites that include

1. only missing data and heterozygotes,
2. missing data, homozygotes for the most frequent allele and one heterozygote,
3. missing data, homozygotes for one ALT allele and one heterozygote.

We include a list of the removed sites in a bed-like file ending with *-includedSites.txt* in the working directory. The file includes information from columns CHROM, POS, and QUAL for the respective sites, and records the (good practice) **record of genotype encoding**.

We include a list of the removed sites in a bed-like file ending with *-omittedSites.txt* in the working directory. The file includes information from columns CHROM, POS, and QUAL for the respective sites, and provides the reason, why the site was omitted. **The file *-omittedSites.txt* must be checked when interpreting polarised sites** to ensure correct site annotation.

4.4 Polyploid data

The same principles apply to higher ploidy datasets. Preparing data in `diem` format for genome polarisation includes these steps:

1. Remove indels.
2. Identify two most frequent alleles for each site.
3. Encode homozygous and heterozygous (mixed) genotypes that include only the two most frequent alleles.

Note that `vcf2diem` only converts diploid datasets at this time, and should not be used to convert polyploid VCF files.

Chapter 5

Understanding genome polarisation output files

Natália Martínková and Stuart J. E. Baird

Genome polarisation (Baird et al., 2023) uses the *diem* algorithm to estimate which allele of a single nucleotide polymorphic marker belongs to which side of a barrier to gene flow based on whole-genome associations. The key result is then the **marker polarity** (whether to read the marker as in the *diem* input file or whether to flip the homozygous state labels $0 \leftrightarrow 2$), **marker diagnostic index** (how relevant the marker is with respect to the barrier to gene flow), and **marker polarity support** (how certain we are regarding whether to flip the state labels or not).

This information is stored in a file *MarkerDiagnosticsWithOptimalPolarities.txt* with columns Marker, newPolarity, DI, and Support. However, the `diem` function generates other outputs, which help to control the genome polarisation analyses and to interpret the results. Some of this information is identical to the function return value. The documentation for the return values can be found by running `?diem`. This vignette explains the outputs saved to files.

5.1 Obligatory outputs

Three output files will be saved to the working directory or to the path specified in the `verbose` argument. Those are:

- MarkerDiagnosticsWithOptimalPolarities.txt,
- HlwithOptimalPolarities.txt, and
- I4withOptimalPolarities.txt.

5.1.1 MarkerDiagnosticsWithOptimalPolarities

The *MarkerDiagnosticsWithOptimalPolarities.txt* is a tab-delimited table with four columns and the number of rows equal to the number of markers across all compartments used to run genome polarisation with **diem**. This is the key results file identifying the marker relevance with respect to the detected barrier to gene flow.

The first column **newPolarity** contains TRUE/FALSE values, specifying which allele in the specific marker belongs to which side of the barrier. When the value is **FALSE**, encoded as 0 in Eq. 22 (Baird et al., 2023), the allele encoded as 0 in its homozygous state in the input file belongs to the barrier side associated with low values of hybrid index of the individuals. When the newPolarity value for a marker is **TRUE**, the homozygous states need to be flipped $0 \leftrightarrow 2$ to correctly associate with the barrier sides. In terms of interpretation, this means that the allele encoded as 2 in its homozygous state is associated with low values of the hybrid index. Which specific alleles these are for any marker can be found in the *includedSites.txt* file from the **vcf2diem**.

The second column **DI** is numeric with values of the diagnostic index of the markers as specified in Eq. 20 (Baird et al., 2023). The diagnostic index is a log likelihood, so its values are always negative. Polarised markers with high diagnostic index characterize the barrier to gene flow. Such markers will tend to be fixed for one allele on either side of the barrier. Conversely, markers with low diagnostic index will have signal orthogonal to the detected barrier to gene flow. They could reflect ancestral genetic variation, standing variation not contributing to the barrier to gene flow, or artifacts such as overmerging of repeated motifs onto the reference.

The third column **Support** is numeric and the values represent polarity support for the markers as given in Eq. 21 (Baird et al., 2023). The support is the gain in likelihood when the correct marker $0 \leftrightarrow 2$ ‘flipping’ is compared to its reverse. High marker support gives confidence that the marker polarity reflects the detected barrier to gene flow. In general, markers with high diagnostic index tend to also have high support.

The fourth column **Marker** is an index of the marker that is successive in the files as they are ordered in the **files** argument in the **diem** call. The value in the Marker column will correspond to the respective index in the file ending with *includedSites.txt* if the input files were generated by the **vcf2diem** function.

5.1.2 HIwithOptimalPolarities

The *HIwithOptimalPolarities.txt* is a tab-delimited one-column matrix of individual hybrid indices with row names corresponding to the individual indices. Note that the hybrid indices are calculated from the polarised genotypes for all individuals found in the input files, not only those specified in the **ChosenInds**

argument of the `diem` function. The hybrid indices are calculated as genome admixture from summaries of polarised genotypes as in Eq. 7 (Baird et al., 2023).

CAUTION: These hybrid indices are calculated *without taking into account the diagnosticity of markers!* As such they may not show barrier signal well at all. Hybrid indices and genome polarisation diagrams should be calculated and plotted after the user decides on a diagnostic index threshold. The ideal threshold will be specific to your dataset. Check **Chapter 7** for guidelines how to calculate it.

If you choose to ignore this warning and include all markers when calculating hybrid indices you will find the range of hybrid indices is small, and centred on 0.5. This is because most genome sites are close to invariant, and so `diem` will flip homozygous state labels $0 \leftrightarrow 2$ at random. A purely random answer will give hybrid index equal to 0.5. You can rescale hybrid indices with a large random influence using Eq. 11 (Baird et al., 2023), so they lie on the interval $[0, 1]$.

5.1.3 I4withOptimalPolarities

The *I4withOptimalPolarities.txt* file contains genome-wide summaries of genomic states for all input genomes (Eq. 4 in (Baird et al., 2023)). It is a tab-delimited table with four columns representing the unknown state (column name “_”) that cannot be encoded as a genotype in terms of the two most frequent alleles, the second column representing homozygote encoding 0, the third column heterozygote encoding 1 and the fourth column homozygote encoding 2. The rows are all individuals in the input files, with row names corresponding to indices.

The 4-genomic state counts can be used to calculate the hybrid index, observed heterozygosity and error rate for all individuals. The equations 7, 9, and 10 (Baird et al., 2023) are implemented in the function `pHetErrOnStateCount` that can be applied to the rows of the *I4withOptimalPolarities*.

However, note that the $^4\mathbf{I}$ in the output was calculated from all markers (see CAUTION in the previous section). We advice the users to preferentially filter their markers based on their diagnosticity after the `diem` analysis and recalculate the hybrid indices and the $^4\mathbf{I}$ from the filtered, polarised genotypes.

5.2 Optional outputs

The `diem` function can output additional files useful for tracking algorithm convergence during iterations and ensuring repeatable initialisation of genome polarisation. The flowchart in Figure 1 specifies what processes generate the files and how the user controls what files will be stored and where.

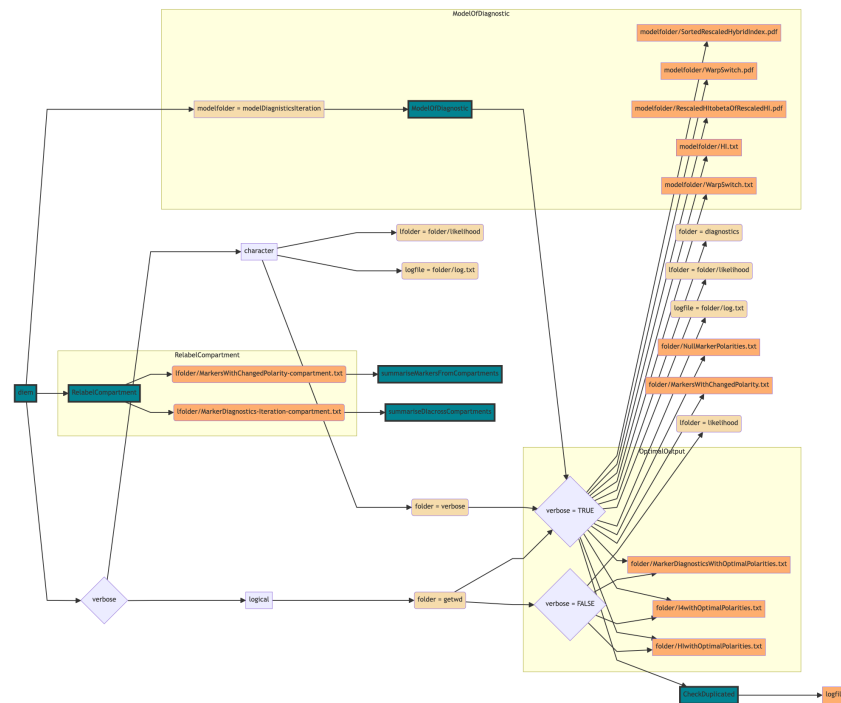


Figure 5.1: **Figure 1**. Flowchart of how to control output files and their location in ‘diem’. Green – functions generating or using the files. Except ‘diem’ all functions are internal. Beige, grey – variable values set by the user (grey) or by internal processes (beige). Orange – stored output files. Yellow rectangles – main processes generating files.

5.2.1 The user perspective

The `verbose` argument in the `diem` function controls the stored output files. The default value `verbose = FALSE` writes three key output files in the current working directory (Figure 1). To locate the files, the current working directory can be found with:

```
getwd()
```

Additionally, the `verbose` argument can trigger a verbose output in two ways.

1. When `verbose` contains a character string with a path to a specific folder, the verbose output will be stored in that location.
2. When `verbose = TRUE`, the verbose output will be stored in the current working directory.

5.2.2 NullMarkerPolarities

The optimal approach to genome polarisation with `diem` is to strip all possible data cues before the analysis. In this way, if `diem` finds a barrier you can be certain it is an intrinsic property of the data. Using random initial marker polarities is central to this approach. We recommend setting the `diem` argument `markerPolarity = FALSE`. The function then generates random polarities for the initiation of the algorithm iterations.

This initial state for `diem` is saved in file *NullMarkerPolarities.txt*. It is a text file the first line describing the contents of the file. The following lines show space-separated TRUE or FALSE values, where each row corresponds to a compartment as they were specified in the `files` argument in `diem`. Because `diem` is a deterministic algorithm, if you start it from the same initial state, you will get precisely the same answer. You can test this by setting `markerPolarity` to be the list from *NullMarkerPolarities*.

```
nullPolarities <- readLines("folder/NullMarkerPolarities.txt")[-1]
nullPolarities <- lapply(
  strsplit(nullPolarities, split = " "),
  as.logical
)
diem(..., markerPolarities = nullPolarities)
```

You can also test the convergence of the algorithm from different initial states by running the same analysis, but each time generating a new random null by setting `markerPolarities = FALSE`.

5.2.3 MarkersWithChangedPolarities

The file *MarkersWithChangedPolarities.txt* is a tab-delimited file with three columns and a variable number of rows. The column **changedMarkers** shows indices of markers across all compartments for which the polarity was changed in the given iteration compared to the marker polarity in the previous iteration. The column **time** gives a time stamp of when the changes were recorded. The column **iteration** specifies the iteration of the algorithm.

Note that here, markers are pooled across all compartments. The information is useful in tracking convergence in postprocessing, and so is not of interest to the casual user. During the analysis run, keeping the information about polarity changes separate for each compartment is computationally more tractable, especially in parallel processing. A set of files specifying the information for each compartment is stored in a *likelihood* folder (Figure 1).

5.2.3.1 MarkersWithChangedPolarities-(compartmentNumber)

The contents of the files is analogical as in the *MarkersWithChangedPolarities.txt*. The difference being that the marker indices are sequential for each compartment.

5.2.3.2 MarkerDiagnostics(iteration)-(compartmentNumber)

The contents of the files correspond to the contents of the *MarkerDiagnosticsWithOptimalPolarities.txt*. Here, the polarities, diagnostic indices and their supports are reported separately for each iteration and for each compartment.

Both sets of compartment-specific temporary files are initially stored regardless of the value in the **verbose** argument, as they are used internally during the analysis. When **verbose = FALSE**, *diem* attempts to delete the *likelihood* folder on exit.

5.2.4 Iteration-specific graphics

The internal `ModelOfDiagnostic` function implements Eq. 19 (Baird et al., 2023) that is the powerful tool to detect the most prominent barrier to gene flow in the data. Seeing how individuals become separated on either side of the barrier provides insights into the algorithm convergence.

5.2.4.1 WarpSwitch files

The *WarpSwitch.pdf* files plot differences in sorted rescaled hybrid indices (Eq. 12) used to identify the barrier to gene flow (Baird et al., 2023). The *WarpSwitch.txt* files then describe what genomic states the ideal diagnostic marker

would have for all included individuals (Eq. 13), and how the ideal 4-genomic state count matrix (Eq. 14) would look like for the first ten individuals.

5.2.4.2 Hybrid index files

The *SortedRescaledHybridIndex.pdf* shows how the differences in hybrid indices between individuals belonging to either side of the barrier become more pronounced with progressing iterations. Note that in order to identify which individuals are placed where, one needs to replicate the figure from hybrid index values in files *HI.txt*.

The files *RescaledHItobetaOfRescaledHI.pdf* show the weight for the hybrid index (Eq. 15) that together with the parameter value in the `diem` argument `epsilon` will inform on how much emphasis the analysis puts on the ideal state with the center of the barrier identified in the WarpSwich files.

The visualisation in the iteration-specific graphics provide a tangible track record of how the theoretical ideals of pure individuals and perfectly diagnostic markers contribute to informing on a barrier to gene flow in a realistic scenario, where there are no pure individuals and no perfectly diagnostic markers.

5.2.5 The log file

The *log.txt* file provides a human-readable overview of the `diem` iterations. During initialisation stage, the log file will list the number of markers and individuals in the analysis run, followed by the top seven rows of the 4-genomic state count matrix ${}^4\mathbf{I}$ with null polarisation, correction for small data (Eq. 5), top seven rows of the ${}^4\mathbf{A}$, which is the ${}^4\mathbf{I}$ matrix corrected for compartment- and individual-specific ploidies and is used to calculate the hybrid index, the top seven rows of the weighted ${}^4\mathbf{V}$ matrix (Eq. 16), and the respective log-likelihood values (Eq. 19).

For subsequent iterations, the log file also quantifies how many markers switch polarities between iterations, and shows the corresponding change to the ${}^4\mathbf{I}$.

Chapter 6

Visualisation of polarised genotypes

Visualisation is a key step in interpreting the results of genome polarisation. The `diemr` package provides several functions for graphical inspection of polarised genotypes, including `plotPolarized` for genome-wide patterns, `plotMarkerAxis` for adding chromosome information, and `plotDeFinetti` for individual genotype composition summaries.

6.1 Plotting polarised genotypes

The function `plotPolarized` displays genotypes in either a rectangular or a circular (iris-style) layout. Rows represent individuals and columns represent single-nucleotide variants (SNVs). Colours correspond to genotype states: by default, purple and green mark the two homozygote classes (0 and 2), yellow marks heterozygotes (1), and white indicates missing data (`_`).

The polarised genotypes must first be imported with correct site polarity. The polarity will be in memory in element `res$markerPolarity` described in (Chapter 2) or in the obligatory output file *MarkerDiagnosticsWithOptimalPolarities.txt* (Chapter 5.1.1) in the column `newPolarity`.

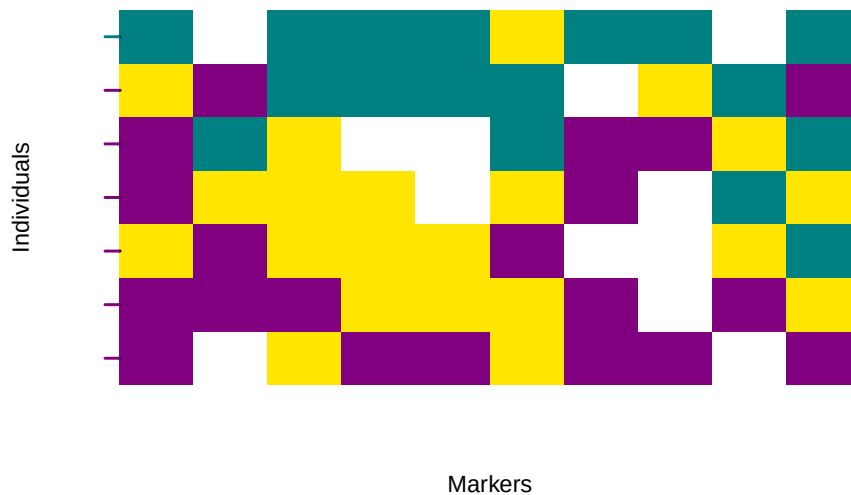
```
# Import polarised genotypes
gen <- importPolarized(
  file = system.file("extdata", "data7x10.txt", package = "diemr"),
  changePolarity = c(TRUE, FALSE, TRUE, TRUE, FALSE, FALSE, TRUE, FALSE, FALSE, TRUE),
  ChosenInds = 1:7
)
```

```
# Calculate individual hybrid indices from the imported genotypes
HI <- hybridIndex(gen)
```

6.1.1 Rectangular plot

The rectangular layout is useful for viewing the order of markers along the genome and for comparing individual hybrid indices along the vertical axis. This is the default plot, and uses the matrix of polarised genotypes and the numeric vector of individual hybrid indices.

```
plotPolarized(genotypes = gen, HI = HI, useRaster = FALSE)
```



Each individual is plotted in order of increasing hybrid index, from bottom to top. A colour transition across the plot reveals the genomic gradient between parental sides.

6.1.1.1 Adding chromosome information

If the input data were generated from a sorted VCF file (Chapter 4), marker positions are already available. The file `*-includedSites.txt` produced by `vcf2diem` stores physical coordinates for each marker and can be used to draw

chromosome names and position ticks beneath the genotype matrix. This file links plotted markers to their genomic locations.

The following example uses a snippet of SNV data with *Myotis* bat diversity (Harazim et al., 2021).

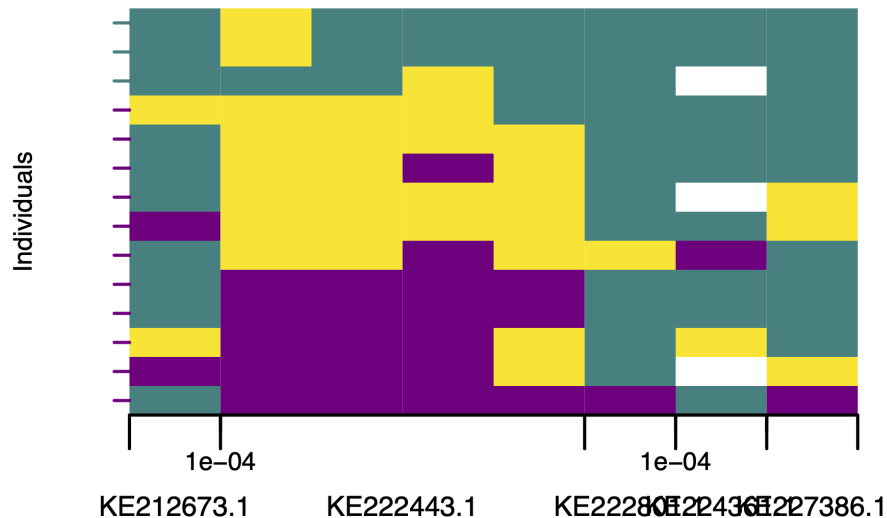
```
# Prepare input data
myo <- system.file("extdata", "myotis.vcf", package = "diemr")
vcf2diem(SNP = myo, filename = "myo")
inds <- 1:14

# Polarise genotypes assuming diploid data
res <- diem("myo-001.txt", ChosenInds = inds, ploidy = FALSE)

# Import polarised genotypes
gen <- importPolarized("myo-001.txt",
  changePolarity = res$markerPolarity,
  ChosenInds = inds)

# Calculate hybrid indices
HI <- hybridIndex(gen)

# Plot polarised genotypes with the marker axis
plotPolarized(
  genotypes = gen,
  HI = HI,
  addMarkerAxis = TRUE,
  includedSites = "myo-includedSites.txt",
  tickDist = 100
)
```



If the default marker axis needs adjustments, such as here, where accession numbers overlap, use `plotMarkerAxis` directly. This returns an `axisInfo` list so you can modify labels before redrawing. `plotMarkerAxis` also accepts additional graphical arguments. For the full list of supported graphical arguments, check the function help with `?plotMarkerAxis`.

```
# Draw plot without axis
plotPolarized(
  genotypes = gen,
  HI = HI,
  addMarkerAxis = FALSE,
  xlab = ""
)

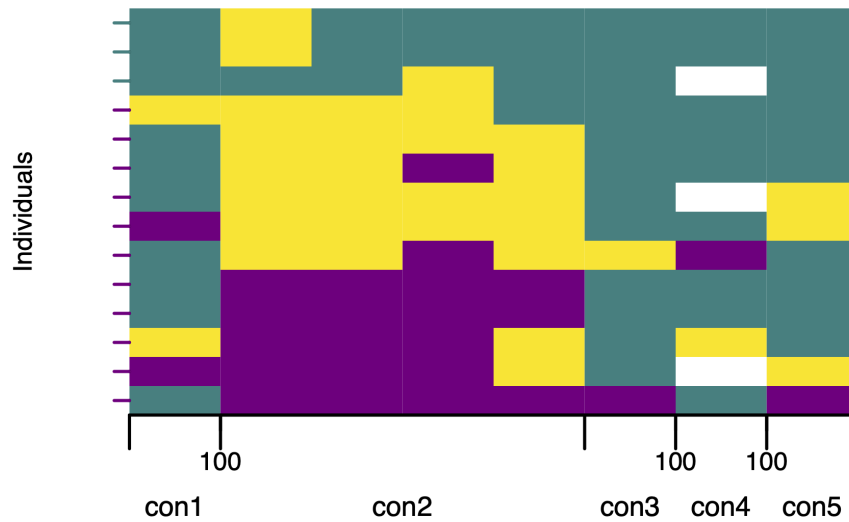
# Draw marker axis, and capture axis information
axisInfo <- plotMarkerAxis(includedSites = "myo-includedSites.txt", tickDist = 100)
```

The `axisInfo` list will now contain five elements with plotting data. Let's shorten the contig names and convert the tick labels to base pairs (default is to display millions of base pairs). This list can then be used for plotting directly, without the need to recalculate the position of the axis elements from the `*-includedSites.txt` file.


```
print(axisInfo)
# $CHROMbreaks
# [1] 0.5 1.5 5.5 6.5 7.5 8.5
#
# $CHROMnamesPos
# [1] 1.0 3.5 6.0 7.0 8.0
#
# $CHROMnames
# [1] "KE212673.1" "KE222443.1" "KE222801.1" "KE224361.1" "KE227386.1"
#
# $ticksPos
# [1] 1.5 6.5 7.5
#
# $ticksNames
# [1] 1e-04 1e-04 1e-04

# Update list elements
axisInfo$CHROMnames <- c("con1", "con2", "con3", "con4", "con5")
axisInfo$ticksNames <- c(100, 100, 100)

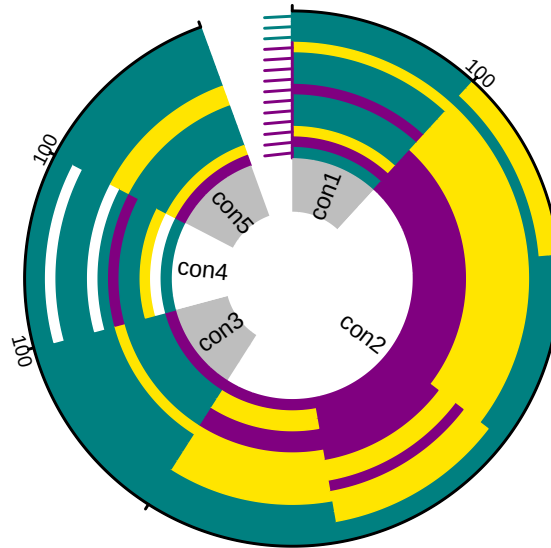
# Plot polarised genotypes with updated marker axis labels
plotPolarized(
  genotypes = gen,
  HI = HI,
  addMarkerAxis = FALSE,
  xlab = ""
)
plotMarkerAxis(axisInfo = axisInfo)
```



6.2 Circular (iris) plot

The circular layout displays the same information radially, emphasising the continuity of ancestry along chromosomes. Individuals are ordered from the centre (lowest hybrid index) to the periphery (highest).

```
plotPolarized(genotypes = gen, HI = HI, type = "circular")
plotMarkerAxis(axisInfo = axisInfo,
  labels.facing = "in",
  major.tick.length = 1
)
```



The marker axis in circular plots is drawn analogically to rectangular plots (Chapter 6.1.1.1), but chromosome names and tick marks are shown as separate elements. Chromosome names appear in an inner track with alternating white and grey shading, while tick marks showing distances along each chromosome are plotted on the outer rim of the genotype rings.

This layout is compact for displaying genomes of many individuals and highlights symmetry or asymmetry between parental contributions. However, circular plotting of large datasets (more than 10^5 markers) can be slow. To monitor progress, set `showProgress = TRUE` in `plotPolarized`. This prints the percentage of plotted individuals to the standard output during rendering.

Chapter 7

Filtering sites by diagnostic index

After obtaining polarised genotypes and diagnostic indices from previous steps, the next step is to identify markers that most clearly distinguish the two sides of the barrier. This chapter demonstrates how to use the diagnostic index for that purpose. A note in the Chapter 2 cautioned against using whole-genome summaries for downstream analyses. This chapter expands the *Quick start* introduction.

During genome polarisation, each site receives a likelihood-based **diagnostic index (DI)** that quantifies its contribution to distinguishing the two sides of the barrier. Sites with low DI are weakly informative and act as noise; they inflate apparent heterogeneity and reduce clarity in hybrid index (HI) plots. Filtering sites by DI allows extraction of informative genomic markers that:

- reduce noise from standing genetic variation,
- produce steeper clines in HI profiles,
- enhance contrast between groups in genotype heatmaps,
- stabilise window-based summaries.

However, filtering is a trade-off. Retaining too few sites can oversimplify the genomic pattern, while keeping all markers may obscure structure within background noise. No universal DI threshold exists; the optimal cutoff depends on genetic variation within the dataset and the divergence captured by the barrier to gene flow. Therefore, inspect the results visually to decide.

The DI values are exported for all sites, and are either available in the standard output from `diem` in an element DI in column DI or in the output file *MarkerDiagnosticsWithOptimalPolarities.txt* (Chapter 5.1.1). This example will

demonstrate DI-filtering from the file. Let's first display polarised genotypes without filtering sites by their diagnostic index.

Full code to prepare data to execute the examples in this chapter

```
library(diemr)

dat <- system.file("extdata", "testBarrier2.txt", package = "diemr")
inds <- 1:6

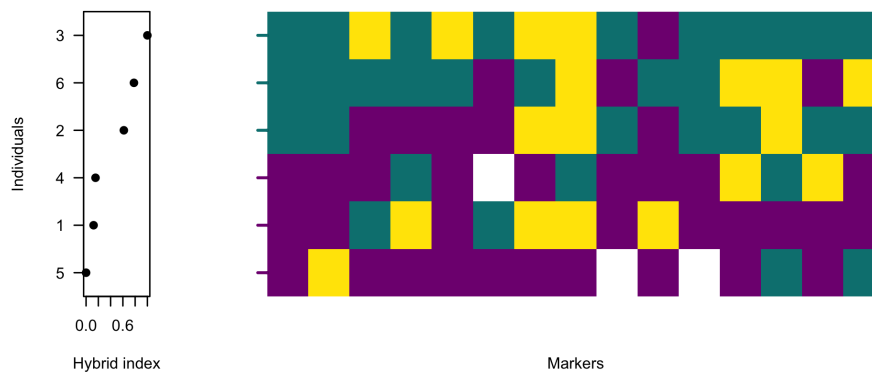
res <- diem(files = dat, ChosenInds = inds)

# read the marker diagnostics file. read.table is possible,
# but for large genomic datasets, fread from the package
# data.table is more efficient
DI <- data.table::fread("MarkerDiagnosticsWithOptimalPolarities.txt",
  header = TRUE, data.table = FALSE
)

# import polarised genotypes for the diagnostic markers
gen <- importPolarized(
  system.file("extdata", "testBarrier2.txt", package = "diemr"),
  changePolarity = DI$newPolarity,
  ChosenInds = 1:6,
  ChosenSites = "all"
)

# calculate hybrid indices from the filtered genotypes
HI <- hybridIndex(gen, rescale = TRUE)

# plot HI and all genotypes
layout(matrix(c(1,2,2,2,2), nrow = 1))
indOrder <- order(HI)
# plot hybrid indices
plot(HI[indOrder], 1:length(HI),
  xlab = "Hybrid index", ylab = "Individuals", axes = FALSE,
  ylim = c(0.5, length(HI) + 0.5), pch = 19,
  yaxs = "i", xpd = NA)
# add axes with ordered individual labels
axis(1)
axis(2, at = 1:length(HI), labels = inds[indOrder], las = 1)
box()
# plot genotypes
plotPolarized(gen, HI = HI, ylab = "")
```



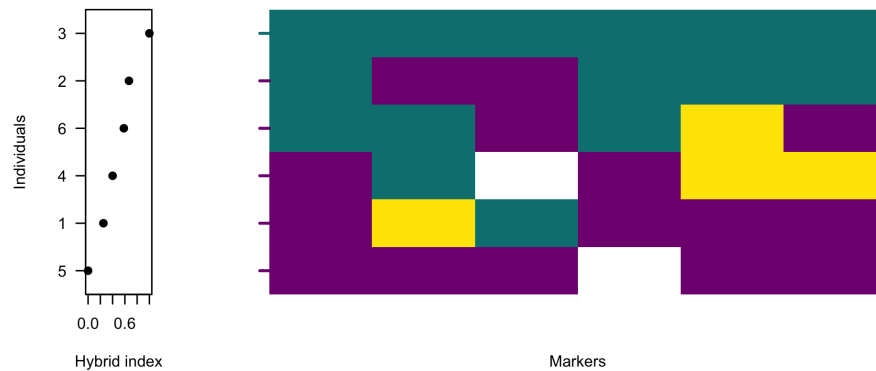
The function `hybridIndex` estimates the proportion of alleles belonging to one side of the barrier across all polarised sites for each individual, automatically accounting for missing data. `plotPolarized` orders individuals by their hybrid index. Displaying the hybrid indices alongside a heatmap of polarised genotypes provides a visual guide to the quantitative change in genome admixture.

The figure shows high genetic variability across the data. Next, let's plot only 40% of markers with the highest diagnostic index.

```
# keep the top 40% most diagnostic sites
threshold <- 0.6
whichMarkers <- DI$DI >= quantile(DI$DI, probs = threshold)

# calculate hybrid indices from the filtered genotypes
HI2 <- hybridIndex(gen[, whichMarkers], rescale = TRUE)

# plot HI and all genotypes
layout(matrix(c(1,2,2,2,2), nrow = 1))
indOrder2 <- order(HI2)
# plot hybrid indices
plot(HI2[indOrder2], 1:length(HI2),
     xlab = "Hybrid index", ylab = "Individuals", axes = FALSE,
     ylim = c(0.5, length(HI) + 0.5), pch = 19,
     yaxs = "i", xpd = NA)
# add axes with ordered individual labels
axis(1)
axis(2, at = 1:length(HI2), labels = inds[indOrder2], las = 1)
box()
# plot genotypes
plotPolarized(gen[, whichMarkers], HI = HI2, ylab = "")
```



7.1 Selecting the filtering threshold

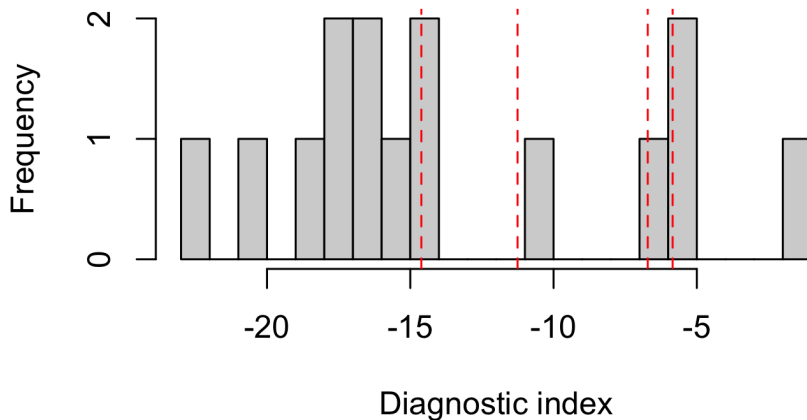
With the hybrid index being a proportion of alleles belonging to one side of barrier to gene flow, typical outcome of filtering SNVs by DI is a set of informative markers that shows **tighter clustering** of individuals near 0 and 1, and a steeper, interpretable gradient across hybrids. However, emphasizing the genomic differences by selecting markers that approach being homozygous on one side of the barrier can have an undesirable effect of highlighting the influence of outliers. These might be inbred individuals, samples with low sequencing coverage or mapping quality or divergent samples informing of a biologically-relevant mechanism. Such individuals are rarely the target of admixture studies, yet they can disproportionately influence the *diem* algorithm and appear as spurious barriers in heavily filtered datasets. To avoid this, the user has two options.

1. To remove the outlier individuals from the analyses using the `ChosenInds` argument.
2. To find the DI filtering threshold that displays the desired barrier to gene flow before it becomes overloaded with outlier pull.

The first option is the cleanest, but both are not mutually exclusive. Let's use the previous example to explore a range of threshold values and their effect on the barrier to gene flow.

```
thresholds <- c(0.6, 0.7, 0.8, 0.9)

# inspect distribution of DI values
hist(DI$DI, xlab = "Diagnostic index", n = 20, main = "")
abline(v = quantile(DI$DI, probs = thresholds), col = "red", lty = 2)
```

Plotting the DI distribution helps identify whether the dataset has a natural break or continuous spectrum of diagnostic values, guiding selection of a reasonable cutoff. Often in genomic datasets, the distribution of DI values is unimodal. In such cases, exploring how DI filtering affects hybrid index profiles is a more informative way to select an appropriate threshold.

```
# load HI from the ploidy-aware whole-genome HI in diem output
HI <- hybridIndex("HIwithOptimalPolarities.txt", rescale = FALSE)
indOrder <- order(HI)

layout(matrix(c(1,1,1,2), nrow = 1))
plot(HI[indOrder], pch = 19, ylim = c(0, 1), axes = FALSE,
      xlab = "Individuals", ylab = "Hybrid index"
)
axis(2, las = 1)
axis(1, at = 1:length(HI), labels = indOrder)
box()

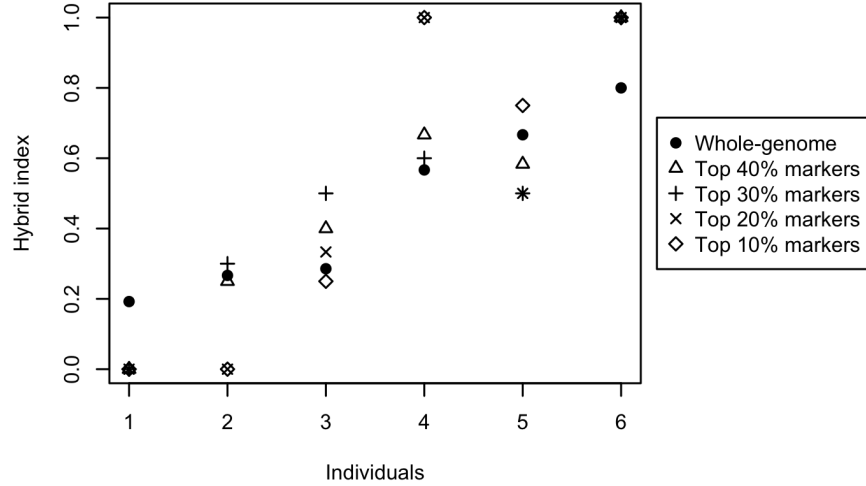
for (threshold in thresholds){
  whichMarkers <- DI$DI >= quantile(DI$DI, probs = threshold)
  HI <- hybridIndex(gen[, whichMarkers], rescale = FALSE)
  points(1:length(HI), HI[indOrder], pch = threshold * 10 - 4)
}

# legend for the comparison of DI thresholds
plot.new()
legend("right", xpd = NA,
```

```

legend = c("Whole-genome", paste0("Top ", 100 * (1 - thresholds), "% markers")),
pch = c(19, thresholds * 10 - 4)
)

```



Based on these plots, the barrier to gene flow progressively increases. Note that the order of individuals was taken from the *HIwithOptimalPolarities.txt* file produced by the `diem` run. These values are based on whole-genome variability taking into account compartment ploidy as it was defined during the analysis. This is the preferred way to explore the influence of alternative threshold values, because the visualisation is also sensitive to how individuals might change positions along the HI slope.

Caution: Because DI depends on allele frequency differences, sample size, and the polarities of all assessed sites, DI values are not directly comparable across independent runs, species pairs, or datasets. Derive cutoff thresholds (for example, top quantiles) within each dataset, and report the number or proportion of retained markers.

7.2 Biological signal insights

Filtering by DI is not mandatory, but it is an efficient way to reveal the core signal of genome polarisation. Patterns that were previously noisy or ambiguous often become clear once low-information markers are excluded. When reporting results, state your DI quantile cutoff and confirm that qualitative interpretations remain robust across reasonable thresholds.

Diagnostic markers often occur in genomic regions under strong differentiation or selection that maintain alternative alleles between taxa, whereas sites with low diagnostic index are typically typically reflect shared or neutral polymorphisms. Thus, filtering sites by diagnostic index not only increases contrast but also enriches for genomic regions that maintain species boundaries.

Because markers with high DI are not evenly distributed along chromosomes (Jagoš et al., 2025), DI filtering can shift the apparent contribution of different genomic regions to the overall signal. Regions with many fixed differences will remain densely represented, while more homogeneous or highly variable regions may contribute fewer diagnostic markers. When summarising results in windows or genomic compartments, keep this unequal site density in mind.

Chapter 8

Smoothing polarised genotypes

After filtering markers by diagnostic index (Chapter 7), genome polarisation analyses can be mapped to physical coordinates to reveal the genomic landscape of barriers to gene flow. Because SNVs are rarely evenly spaced along chromosomes, the polarised signal can fluctuate sharply between adjacent markers. But these fluctuations might not represent signal change, but confluence with standing genetic variation. While, diagnostic index filtering already removed much of the effect of genetic variation orthogonal to the barrier to gene flow, smoothing along mapped distances helps visualise broad ancestry patterns.

8.1 Smoothing across mapped distances

The `smoothPolarizedGenotypes` function applies kernel smoothing to the polarised genotype matrix, using physical distances between markers rather than their rank order. This approach preserves the broad genomic signal while suppressing local irregularities caused by uneven SNV spacing.

8.1.1 Mapping diagnostic markers

The chromosomal coordinates of diagnostic markers are stored in the output file *-includedSites.txt*, created during VCF conversion by the function `vcf2diem`. This file lists all sites retained for polarisation and includes columns CHROM, POS, QUAL from the VCF file and allele0 and allele2 indicating alleles represented by the respective numbers when homozygous.

The positional information needed for smoothing can be extracted directly from this file. The function `rank2map` converts the ranked order of sites in the polarised genotype matrix into their corresponding chromosomal coordinates based on the `-includedSites.txt` file. While it is called internally by `smoothPolarizedGenotypes`, exploring its output helps understand how physical mapping guides the smoothing process.

Full code to prepare data to execute the examples in this chapter

```
library(diemr)

dat <- system.file("extdata", "myotis.vcf", package = "diemr")

vcf2diem(SNP = dat, filename = "myo", requireHomozygous = FALSE)

set.seed(54869)
res <- diem("myo-001.txt", ChosenInds = 1:14)

gen <- importPolarized("myo-001.txt",
  changePolarity = res$markerPolarity,
  ChosenInds = 1:14
)

HI <- hybridIndex(gen)
```

```
# read the includedSites file
bed <- readIncludedSites("myo-includedSites.txt")

# generate mapping of ranks to genome positions
windows <- rank2map("myo-includedSites.txt", windowSize = 150)

# show how window ranges correspond to sites
cbind(windows, bed)
```

#	1	2	CHROM	POS	QUAL	allele0	allele2
# 1	1	1	KE210828.1	60	.	A	G
# 2	2	2	KE210828.1	171	.	G	T
# 3	3	3	KE212673.1	84	.	A	T
# 4	4	4	KE214857.1	90	.	A	G
# 5	5	8	KE222443.1	134	.	G	A
# 6	5	8	KE222443.1	156	.	T	G
# 7	5	9	KE222443.1	171	.	T	G
# 8	5	9	KE222443.1	183	.	C	T
# 9	7	9	KE222443.1	244	.	C	G
# 10	10	10	KE222801.1	100	.	G	A
# 11	11	14	KE224361.1	120	.	G	A
# 12	11	14	KE224361.1	132	.	G	C

```
# 13 11 14 KE224361.1 175 . A C
# 14 11 14 KE224361.1 179 . G T
# 15 15 15 KE227386.1 153 . A G
```

Each entry in `windows` corresponds to one marker in the *myo-includedSites.txt* file and shows a range of sites that are within the desired `windowSize`, where the window is centered at the reference site. In practice, `rank2map` looks for half the `windowSize` in either direction from the evaluated site, respecting the affinity to unique `CHROM` values. For genomes composed of multiple scaffolds or chromosomes, smoothing should be performed separately for each element, and `rank2map` ensures that the windows are set up with respect to `CHROM` values.

8.1.2 Visual comparison of raw and smoothed results

Smoothing should only be applied to mapped data. Using it on unordered or scaffold-level datasets can distort the genomic signal.

For smoothing to produce meaningful results, the sites in the original VCF file must be sorted by their genomic coordinates. This is an important consideration of the data prior to `vcf2diem` conversion. See Chapter [?\(vcf2diem\)](#).

The function `smoothPolarizedGenotypes()` then applies a **Laplace-kernel weighted mode** (Chapter [?\(Laplace\)](#)) to neighbouring markers within a defined window.

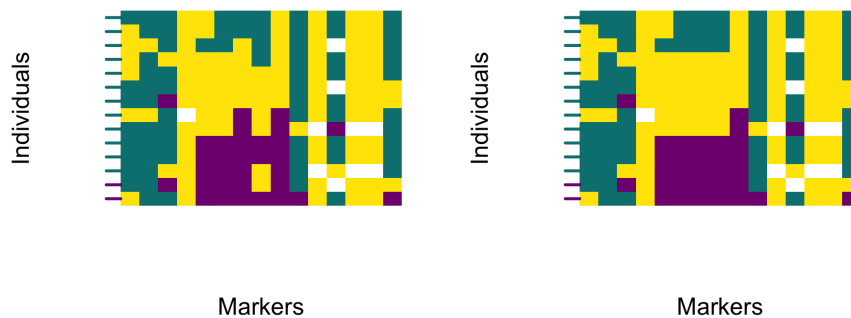
```
# smooth the polarised genotypes using a 150 bp window
genSmooth <- smoothPolarizedGenotypes(
  genotypes = gen,
  includedSites = "myo-includedSites.txt",
  windowSize = 150
)
```

The function returns a genotype matrix with the same dimensions and site order as the input, but with smoothed genomic states. These can be used directly in `plotPolarized()` or in hybrid-index calculations just like raw genotypes.

The window size is specified in base pairs and should match the expected local scale of linkage. Because recombination rates differ among taxa and genomic regions, it is best to compare several window values empirically. Smaller windows follow local fluctuations more closely, whereas larger windows yield a smoother signal that can merge nearby ancestry blocks. The goal is to retain broad ancestry patterns while avoiding over-smoothing.

The example below compares raw and smoothed genotype visualisations. The smoothing does not change overall polarity—it only reduces short-range noise introduced by individual SNVs.

```
par(mfrow = c(1, 2))
plotPolarized(gen, HI)
plotPolarized(genSmooth, HI)
```



The smoothed profile follows the same general trajectory as the raw data but removes fine-scale irregularities. The resulting heatmap highlights broad ancestry transitions and provides a clearer view of barriers to gene flow along chromosomes.

8.2 Interpretation and downstream use

Smoothing algorithm implemented in *diemr* since version 1.4.2 uses a truncated Laplace kernel to calculate weighted mode genomic state for each smoothed site. The Laplace kernel gives the highest weight to nearby markers while gradually down-weighting distant ones, producing smooth but localised transitions in the polarised signal. The truncation limits the influence of distant loci so that ancestry shifts remain aligned with finite physical genomic distance.

Mapping and smoothing improve interpretability of *diem* output in three ways:

1. stabilising local genomic states and reducing noise caused by marker spacing,
2. expressing barrier strength and ancestry transitions in physical coordinates, and
3. facilitating integration with genomic context such as recombination rate, gene density, or structural features.

When reporting smoothed results, always specify the window size and kernel type used. Smoothing clarifies broad-scale patterns but should not replace marker-level interpretation.

Methodological explanation of the Laplace kernel smoothing

8.2.1 Methodological explanation of the Laplace kernel smoothing

The smoothing algorithm applies a truncated Laplace kernel over physical distance and assigns each site the weighted modal genomic state within the window. This can be described as **Laplace-kernel weighted mode smoothing**.

Smoothing is applied **per chromosome**. For each individual i and focal site j at position x_j , we consider all neighbouring sites within a symmetric window of half-width `windowSize`/2 base pairs. Rather than averaging genotype values, the algorithm computes a **kernel-weighted vote** over discrete genomic states $s \in \{0, 1, 2\}$ (unknown genomic state $_$, homozygous genotypes of two most frequent alleles denoted as 0 and 2, and heterozygotes as 1, chapter 3) and assigns the **state with the highest weighted support** to the focal site.

The goal is to reduce short-range noise caused by uneven marker spacing and stochastic variation, while preserving discrete genomic states that reflect linkage blocks and recombination.

The smoothing method considers a **centred physical window** of width `windowSize` (bp), i.e. positions x with

$$x \in \left[x_j - \frac{\text{windowSize}}{2}, x_j + \frac{\text{windowSize}}{2} \right].$$

Within that window, neighbouring markers “vote” for their discrete genomic states $s \in \{0, 1, 2\}$ with **Laplace weights** that decay with distance from the focal site. The smoothed value at j is the **weighted mode** (state with the largest total weight).

It is important that the smoothing is applied to physical distances between SNVs, so the `rank2map` internally computes the **start and end site indices** whose positions fall inside the centred interval for site j .

8.2.1.1 Implemented kernel, truncation and scaling

Let $d = x - x_j$ be the **centred physical distance** (bp) from the focal site. The kernel is only ever evaluated inside the centred window; weights for positions outside are never computed or used.

Define the **Laplace scale** b as a function of `windowSize`:

$$b = \frac{\text{windowSize}}{2 \ln 20}.$$

This choice guarantees that a two-sided Laplace distribution truncated at $\pm b \ln 20$ retains 95% of its mass (because $\Pr(|X| > a) = e^{-a/b}$, so with $a = b \ln 20$, the tail is $1/20 = 0.05$).

The code evaluates integer-spaced centred offsets

$$x \in -\lfloor \frac{\text{windowSize}}{2} \rfloor, \dots, -1, 0, 1, \dots, \lfloor \frac{\text{windowSize}}{2} \rfloor$$

and assigns **truncated Laplace weights**:

$$w_x = \frac{10}{19} \exp\left(-\frac{|x|}{b}\right), \quad \text{for } |x| \leq \frac{\text{windowSize}}{2}.$$

Notes on scaling and normalisation:

- The constant 10/19 is an explicit global scaling applied to all in-window weights (see **truncatedLaplace**); it is not a per-site normalisation.
- Because the algorithm selects a **weighted mode**, only relative weights matter; any common multiplicative constant cancels.
- There is no per-window re-normalisation; the code compares **sums of weights** across states directly.

8.2.1.2 Weighted-mode smoothing

For each focal site j and individual i , we compute **state-specific support** by summing weights over all neighbours within the centred window whose genotype equals that state:

$$S_s(i, j) = \sum_{k \in \text{window}(j)} \begin{cases} w_{(x_k - x_j)}, & \text{if } g_{i,k} = s, \\ 0, & \text{otherwise.} \end{cases}$$

The **smoothed state** is the argmax:

$$\tilde{g}_{i,j} = \arg \max_{s \in \{0,1,2\}} S_s(i, j).$$

8.2.1.3 Tie-breaking (exact order used)

If two or more states tie, internal function **unbiasedWeightedStateChoice** breaks ties in this strict sequence:

1. **Higher total weight** $S_s(i, j)$ (the primary criterion).
2. **Higher single-occurrence weight** for the state (i.e. the largest w among markers supporting that state).

3. **Lexicographic order** of the state label (with numeric states treated as character, effectively "0" < "1" < "2").

This is the complete list; there is **no** preference for the focal site's own genotype unless it wins under the rules above.

Chapter 9

Ploidy-aware genome polarisation

Genome polarisation with the *diem* algorithm was developed for biallelic genomic sites, where each site contributes one of four possible states: an undetermined genomic state or missing data, homozygous for one allele, heterozygous, or homozygous for the alternative allele. Many biological datasets, however, contain genomic **regions of different ploidy** within the same individual. These include sex chromosomes, pseudo-autosomal regions, organellar genomes, and other genomic regions with reduced or increased ploidy.

The *diem* model of diagnostics assumes that the information contributed by a site scales with the number of allele copies sampled per individual (Baird et al., 2023). Genomic compartments that are haploid or hemizygous therefore contribute less information per site than diploid regions. Treating all sites as diploid can bias likelihood estimates and hybrid index calculations unless ploidy differences are explicitly accounted for, particularly in cases where diagnostic markers accumulate in non-diploid genomic regions.

This chapter describes how `diemr` handles ploidy-aware genome polarisation and how hybrid indices can be calculated consistently across compartments of different ploidy.

9.1 Mapping compartments and their ploidy

A *compartment* is a subset of sites for which we can specify ploidy for each of the analysed individuals. For example:

- autosomes: ploidy = 2 for all individuals,

- X chromosome: ploidy = 2 for XX, 1 for XY,
- mitochondria: ploidy = 1,
- Y chromosome: ploidy = 0 or 1 depending on sex.

We use the term *compartment* rather than *chromosome* because compartments are defined by ploidy regime, not by chromosomal identity. Moreover, when using parallel processing in *diem*, the grain of parallelisation is a compartment. Parallelisation is more efficient (less overhead) if compartments have similar sizes, which is unlikely when each compartment corresponds to a chromosome.

The *diem* algorithm estimates an optimal polarity for each site. Individual ploidy enters through the compartment definition, which specifies how many allele copies are expected per individual for the sites in that compartment and therefore how sites contribute to the likelihood and to the hybrid index.

Compartment ploidy is a required argument in *diem* analysis, and site polarities are inferred from the model that is ploidy-aware. The compartment and ploidy mapping is introduced with a worked example in Chapter 3.1.

Full code to prepare data to execute the examples in this chapter

```
library(diemr)

filepaths <- c(system.file("extdata", "testBarrier.txt", package = "diemr"),
               system.file("extdata", "testBarrier2.txt", package = "diemr"))

ploidies <- list(rep(2, 6),
                c(2, 1, 2, 2, 1, 1))

samples <- 1:6

CheckDiemFormat(files = filepaths,
                 ChosenInds = samples,
                 ploidy = ploidies,
                 quiet = TRUE)
# File check passed: TRUE
# Ploidy check passed: TRUE

# Set random seed for reproducibility
set.seed(39583782)

# Run diem
res <- diem(files = filepaths,
            ploidy = ploidies,
            markerPolarity = FALSE,
            ChosenInds = samples,
            nCores = 1)
```

```
# Import polarised genotypes
gen <- importPolarized(file = filepaths,
                      changePolarity = res$markerPolarity,
                      ChosenInds = samples)
```

9.2 Hybrid index under varying ploidy

In *diem*, the hybrid index is derived from genomic state summaries. For each individual, polarised genotypes are first summarised into counts of the four genomic states (undetermined, 0, 1, 2), forming a $^4\mathbf{I}$ genomic state count vector. Hybrid indices are then calculated from the $^4\mathbf{I}$ genomic state summaries, and not directly from per-site genotypes.

When sites are partitioned into compartments with different ploidy, genomic state counts are first computed separately for each compartment. These compartment-wise $^4\mathbf{I}$ summaries are then converted to allele count summaries by multiplying each individual's compartment counts by the corresponding compartment ploidy. The resulting $^4\mathbf{A}$ allele count matrices are summed across compartments to obtain a single ploidy-aware genomic state summary per individual.

The hybrid index for an individual is calculated from this combined $^4\mathbf{A}$ summary as the proportion of alternative alleles among all called alleles, excluding undetermined states. This procedure ensures that compartments with lower ploidy contribute proportionally fewer allele copies.

This construction allows hybrid index values to be compared consistently within datasets that contain genomic regions of different ploidy.

9.3 Implementation

From *diemr* 1.5.2, the function `hybridIndex` automatically performs ploidy-aware calculations when paths to input data in *diem* format, a vector of polarities for all sites, and a list with ploidy vectors for all individuals and all compartments are provided. This gives identical result to hybrid indices in the output file *HIwithOptimalPolarities.txt*. Hybrid indices calculated without supplying compartment ploidy are implicitly treated as diploid and will generally differ.

```
# Hybrid indices from diem run
HI1 <- hybridIndex("HIwithOptimalPolarities.txt")

# Calculated ploidy-aware hybrid indices
```

```

HI2 <- hybridIndex(x = filepaths, ploidy = ploidies,
  changePolarity = res$markerPolarity)

# Calculated hybrid indices without considering ploidy
HI3 <- hybridIndex(gen)

all.equal(HI1, HI2)
# [1] TRUE

all.equal(HI2, HI3)
# [1] "Mean relative difference: 0.01367976"

```

Adjusting the hybrid indices according to compartment ploidies is especially important when filtering for diagnostic markers (Chapter 7) results in compartments with different ploidies including many diagnostic markers.

```

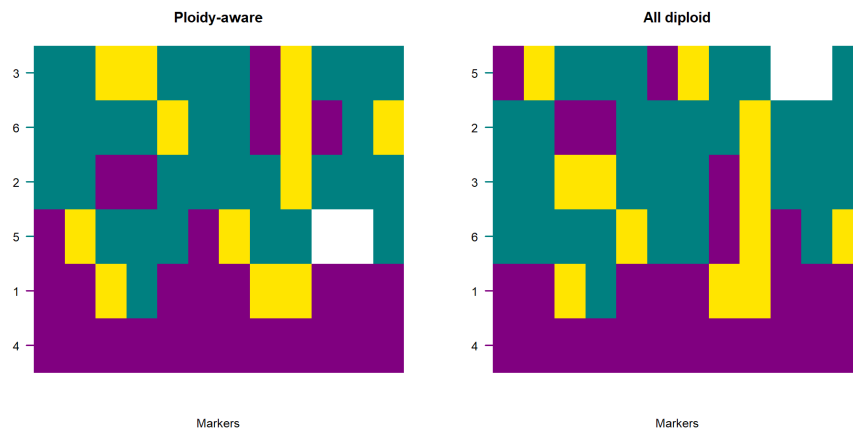
# Select top 40% of the most diagnostic markers
whichMarkers <- res$DI$DI >= quantile(res$DI$DI, prob = 0.6)

hi2 <- hybridIndex(x = filepaths, ploidy = ploidies,
  changePolarity = res$markerPolarity,
  ChosenSites = whichMarkers)

hi3 <- HI3 <- hybridIndex(gen, ChosenSites = whichMarkers)

# plot with ploidy-aware hybrid indices and without ploidy information
par(mfrow = c(1, 2))
plotPolarized(gen[, whichMarkers], HI = hi2, labels = samples, main = "Ploidy-aware")
plotPolarized(gen[, whichMarkers], HI = hi3, labels = samples, main = "All diploid")

```



9.4 Interpretation considerations

Ploidy-aware hybrid indices behave identically to diploid indices when comparing individuals within a dataset—values remain constrained to $[0, 1]$ and reflect the proportion of ancestry belonging to the chosen side of the barrier. However, when regions vary strongly in ploidy (e.g., autosomes vs X vs Y):

- haploid regions produce allele proportions with lower sampling variance per site,
- compartments with many sites dominate the genome-wide mean,
- the influence of mitochondria or sex chromosomes depends strictly on their diagnostic marker count, not their biological magnitude.

When comparing datasets with different compartment definitions or different numbers of markers per compartment, interpretation must therefore focus on **within-dataset structure**, not absolute values across analyses.

Bibliography

- Baird, S. J. E., Petružela, J., Jaroň, I., Škrabánek, P., and Martínková, N. (2023). Genome polarisation for detecting barriers to gene flow. *Methods in Ecology and Evolution*, 14:512–528.
- Harazim, M., Piálek, L., Pikula, J., Seidlová, V., Zukaľ, J., Bachorec, E., Bartonička, T., Kokurewicz, T., and Martínková, N. (2021). Associating physiological functions with genomic variability in hibernating bats. *Evolutionary Ecology*, 35:291–308.
- Jagoš, F., Baird, S. J., Harazim, M., and Martínková, N. (2025). Eroding species barriers: hybridising canids remain distinct around centromeres. *bioRxiv*, page 2025.03.26.645431.